UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
Departamento de Computação – DC
Bacharelado em Ciência da Computação
Disciplina: Sistemas Distribuídos
Prof.: Paulo Henrique Lopes Silva

Prática Offline 1

1. Nomeação em Sistemas Distribuídos: Conceitos Fundamentais e Aplicações.

- Em qualquer sistema distribuído, há recursos (processos, arquivos, serviços, dispositivos, nós, objetos remotos) que precisam ser identificados e localizados.
- O mecanismo que permite isso é o **sistema de nomeação** ***(naming system)***.
- **Nomeação** é o processo de atribuir nomes simbólicos a recursos distribuídos e permitir que esses nomes sejam traduzidos em endereços ou identificadores reais.
- Em outras palavras:
    - **Nome**: representação simbólica compreensível (ex.: "servidor1", "db.local").
        - Um rótulo simbólico usado para referenciar algo.
    - **Endereço**: representação física (ex.: "192.168.0.10", "10.0.0.15:8080");
        - Localização física ou lógica do recurso.
    - **Identificador**: código único interno (ex.: UUID, hash, *handle*).
        - Um valor único que identifica um recurso, independente da localização.
- Esses três conceitos estão interligados, mas não são a mesma coisa.
    - Um nome pode mudar de endereço ao longo do tempo, e um mesmo endereço pode conter diferentes nomes (ex.: servidor virtualizado).

- ***Binding*** **(ligação).**

    - O *binding* é a associação entre nome, identificador e endereço.
    - Há três tipos principais:
        - **Estaticamente ligado:** Feito uma vez e nunca muda.
            - Ex.: "servidor1": 192.168.0.10" fixo no código.
        - **Ligação tardia *(late binding):*** Feita em tempo de execução, mas raramente alterada.
            - Ex.: Cliente consulta DNS a cada conexão.
        - **Ligação dinâmica *(dynamic binding)*:** Atualizada automaticamente conforme o recurso muda.
            - Serviço em nuvem que muda de IP e se registra de novo.
        - Em sistemas modernos (microsserviços, nuvem, IoT), o ***binding*** **dinâmico** é o mais usado, pois os serviços são elásticos e efêmeros.

- **Objetivos do sistema de nomeação.**

    - **Abstração da localização:**
        - O cliente não precisa saber onde o recurso está, apenas quem ele é.
    - **Transparência de mobilidade:**
        - Se o recurso se mover (mudar de IP ou servidor), o nome continua o mesmo.
    - **Escalabilidade:**
        - O sistema deve suportar bilhões de nomes e resolvê-los eficientemente (ex.: DNS global).
    - **Tolerância a falhas:**
        - O sistema deve funcionar mesmo que partes da infraestrutura falhem.
    - **Desempenho:**
        - Resolução rápida, caches e replicação de nomes para evitar gargalos.

- **Tipos de sistemas de nomeação.**

- **Nomeação plana *(Flat naming)*.**
  - Cada recurso tem um identificador único (ex.: hash, UUID).
  - Sem estrutura hierárquica, busca é feita por difusão ou DHT.
  - Muito comum em sistemas P2P (Chord, Pastry, Kademlia).
  - Exemplo: Cada nó na rede tem um identificador SHA-256 e armazena pares chave → valor.

- **Nomeação hierárquica.**
  - Nomes organizados em níveis (semelhante a diretórios).
  - Permite delegação de autoridade.
  - Base do DNS *(Domain Name System)*.
  - Exemplo:
  - www.instituto.ufersa.br
    - br → nível raiz.
    - ufersa → subdomínio.
    - instituto → unidade.
    - www → servidor web (Tradicionalmente aponta para o serviço HTTP/HTTPS).

- **Nomeação baseada em contexto (ou contexto de nomes).**

  - Um mesmo nome pode significar coisas diferentes em contextos diferentes.
  - Muito usada em sistemas de objetos distribuídos (ex.: *RMI Registry*, *CORBA Naming Service*).

- **Nomeação de conteúdo *(content-based)*.**

  - O nome é derivado do conteúdo (hash de um arquivo ou bloco de dados).
  - Usada em sistemas imutáveis e redes de conteúdo (CDNs, IPFS).
  - Exemplo:
  - O arquivo é identificado por SHA-256("conteúdo").

Questões práticas.

1. Mini-DNS para resolução de nomes (sockets UDP): Implementar um servidor simples que mantém um mapa.

```
Java
"servidor1" → "192.168.0.10"
"servidor2" → "192.168.0.20"
"servidor3" → "192.168.0.30"
"servidor4" → "192.168.0.40"
"servidor5" → "192.168.0.50"
"servidor6" → "192.168.0.60"
"servidor7" → "192.168.0.70"
"servidor8" → "192.168.0.80"
"servidor9" → "192.168.0.90"
"servidor10" → "192.168.0.100"
```

- O cliente pergunta pelo nome e o servidor devolve o endereço.
  - Considere a existência de três clientes requisitantes.
- Depois, simular ***binding dinâmico***: um cliente do tipo registrador atualiza os endereços associados a "servidor1", "servidor4" e "servidor9".
  - O servidor envia a atualização aos clientes requisitantes. Mostrar que os clientes recebem o novo valor.

2. Descoberta de serviços *(service discovery)*: Criar servidores que oferecem um serviço simples (ex.: “Calcular soma”, “Calcular subtração”, “Calcular multiplicação” e “Calcular divisão”).

- Três servidores possuem o serviço de calculadora básica, com as operações listadas (cada qual com seu nome e endereço simulados).
- Um servidor de diretório registra todos os servidores disponíveis.
- Considere três clientes:
    - Um cliente consulta o diretório para descobrir qual servidor está ativo.
    - Faça testes de descoberta com, pelo menos, dois serviços.
- Incluir ***load balancing***:
    - Um cliente pega um endereço entre os registrados, de acordo com uma das técnicas.
        - *Round robin*.
        - *Random*.
        - Número de conexões.

3. Nomeação em P2P (*overlay* simplificado): Desenvolva uma aplicação cliente/servidor *multithread*, com TCP ou UDP, para simular a comunicação de processos (nós) em uma topologia em anel. Considere seis processos chamados de P0, P1, P2, P3, P4 e P5. Considere o seguinte esquema:

- Cada nó tem um identificador numérico.
- O nó sabe quem é seu sucessor e seu antecessor.
- Para encontrar um recurso (ex.: arquivo “X”), um nó repassa a mensagem ao próximo até o responsável ser encontrado.
    - Os arquivos podem ser simulados com strings: arquivo1.ext, arquivo2.ext, arquivo23.ext, arquivo47.ext, etc.
    - A rede armazena 60 arquivos, sendo:
        - P0 com os arquivos de 1 a 10.
        - P1 com os arquivos de 11 a 20.
        - P2 com os arquivos de 21 a 30.
        - P3 com os arquivos de 31 a 40.
        - P4 com os arquivos de 41 a 50.
        - P5 com os arquivos de 51 a 60.
- Esse exercício mostra nomeação distribuída sem servidor central.
- Observações:
    - As mensagens de busca por um “arquivo” podem ser enviadas por qualquer processo.
        - Você pode definir o sentido das mensagens (horário ou anti-horário).
- **Casos de teste:**
    - **Caso 1**: Busca válida.
        - Origem: P0, Arquivo: arquivo35.ext
        - Fluxo: P0 → P1 → P2 → P3
        - Resultado:
            - P0 repassa a P1; P1 e P2 verificam que não possuem e repassam.
            - P3 encontra o arquivo (faixa 31–40) e responde a P0.
            - P0, P1, P2 e P3 registram no log o envio e o recebimento das mensagens.
    - **Caso 2**: Busca inválida e entrada inválida.
        - Origem: P1 | Entrada: arquivo99.ext (ou fora do intervalo 1–60).
        - Resultado:
            - O sistema rejeita o comando sem travar o servidor nem propagar mensagens no anel.
            - Imprime no console: Erro na entrada de dados. Tente outra vez!
        - Origem: P4 | Entrada: arquivoABC.ext.
        - Resultado:
            - O sistema rejeita o comando sem travar o servidor nem propagar mensagens no anel.
            - Imprime no console: Erro na entrada de dados. Tente outra vez!
- Sobre as mensagens:
    - As mensagens podem ser de tipos primitivos ou compostos.

- Cada processo deve armazenar e imprimir na tela um log de mensagens recebidas e enviadas.

4. CDN

Uma *Content Delivery Network* (CDN) é uma infraestrutura distribuída composta por servidores estrategicamente posicionados (*Edge Servers* ou *Points of Presence - PoPs*) interligados pela internet. O objetivo fundamental é aproximar o conteúdo do usuário final, reduzindo a latência, otimizando o consumo de banda e aumentando a resiliência do serviço.

Algumas aplicações no mundo real:

- *Streaming* de vídeo e áudio: Plataformas como Netflix, YouTube e Spotify utilizam CDNs para entregar pequenos blocos de mídia (ex: pedaços de arquivos .ts ou .m4s) com baixa latência, evitando *buffering*.
- Distribuição de arquivos grandes (Downloads): Atualizações de jogos (*Steam*, *PlayStation Network*), ISOs de sistemas operacionais e instaladores de software usam a infraestrutura distribuída para suportar picos massivos de tráfego.
- Aceleração de web sites estáticos e dinâmicos: Sites de notícias e e-commerce armazenam imagens, arquivos CSS e JavaScript na borda para garantir carregamento instantâneo das páginas.
- Segurança e mitigação de ataques (DDoS): Ao absorver milhões de requisições por segundo na borda, a CDN impede que ataques volumétricos alcancem o servidor de origem.

4.1. Simulação: Arquitetura da CDN baseada em sockets.

- Para simular o funcionamento interno de uma CDN com sockets, o sistema deve ser dividido em quatro componentes principais:

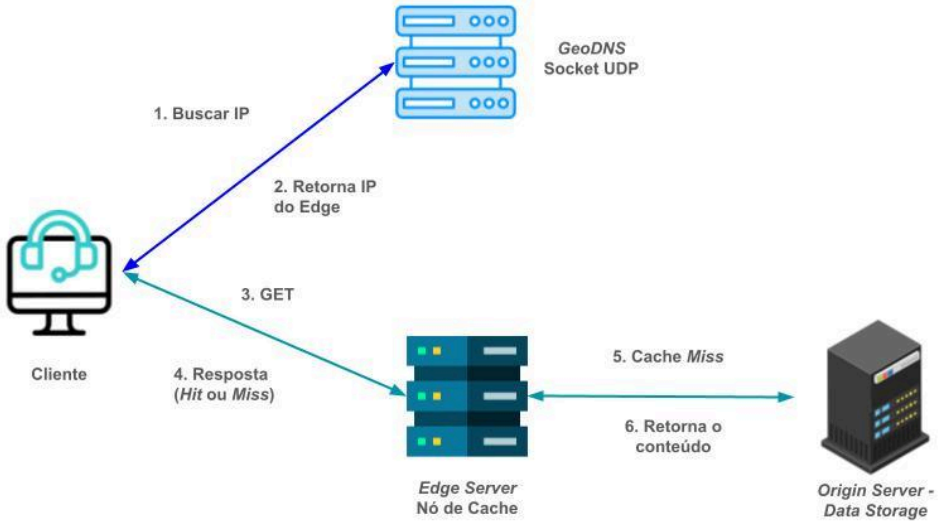


Figura 1: Arquitetura simplificada de uma CDN.

- *Origin Server* (Servidor de Origem).
  - Socket: TCP escutando em uma porta fixa (ex: 8000).
  - Papel: Armazena o banco de dados/arquivos mestre.
  - Lógica: Recebe uma requisição socket, busca o arquivo local no disco e responde o conteúdo bruto.

        - O servidor armazena 1000 arquivos (pode usar a mesma nomenclatura da questão 3).

    - *Edge Servers / Cache Nodes* (Nós de Borda).

        - Socket:
            - Servidor TCP: Escuta requisições dos clientes (ex: portas 8001, 8002, 8003).
            - Cliente TCP: Abre uma conexão com o servidor de origem em caso de *cache miss*.
        - Lógica:
            - Mantém uma tabela hash/dicionário em memória (cache).
            - Ao receber a mensagem do cliente:
                - *Cache hit:* Retorna o dado imediatamente.
                - *Cache miss:* Abre socket com a Origem, baixa o dado, salva na memória/disco local e envia de volta ao cliente.
                - Tamanho do cache: 100 arquivos.

    - *Router / GeoDNS* (Direcionador de Tráfego).

        - Socket: UDP ou TCP escutando numa porta principal (ex: 5353).
        - Lógica:
            - Recebe a solicitação do Cliente: "Onde baixo o arquivo X?".
            - Avalia a localização fictícia do Cliente (enviada no *payload*).
            - Responde ao cliente com o IP/Porta do *Edge Server*.

    - Cliente (Gerador de Tráfego).

        - Socket: Cliente TCP/UDP.
        - Lógica:
            - Consulta o *Router* para descobrir a qual *Edge* se conectar.
            - Estabelece conexão socket TCP com o *Edge* fornecido.

- **Protocolo de aplicação simplificado.**

    - Em vez de usar HTTP completo, você pode definir um protocolo simples baseado em texto/JSON transmitido sobre os sockets TCP:
        - Requisição do Cliente para o Edge:
            - GET /arquivo12.ext
            - Resposta do *Edge* (cache *hit*):
                - 200 OK
                - Cache: HIT
                - Length: 1024
            - Notificação de Invalidação *(Origin -> Edges)*:
                - INVALIDATE /arquivo12.ext

- **Casos de teste.**

    - Para validar o funcionamento da CDN, aplique cenários de teste que demonstrem algumas propriedades de sistemas distribuídos:

        - **Caso 1: Validação de *cache hit* vs. *cache miss*.**
            - Cenário: O cliente solicita o arquivo arquivo20.ext pela primeira vez. Em seguida, solicita o mesmo arquivo novamente.
            - O que observar:
                - A primeira requisição resulta em *cache miss*. O *Edge* abre um socket TCP com a Origem, baixa o arquivo, armazena no cache local e responde ao cliente.

        - A segunda requisição resulta em *cache hit*. O *Edge* responde diretamente ao cliente via socket sem comunicar-se com a Origem.
        - O tempo de resposta na segunda requisição deve ser visivelmente menor.

- **Caso 2: Expiração por TTL *(Time-To-Live)* e consistência.**
    - Cenário: Configure o arquivo no *Edge* com um TTL curto (ex: 10 segundos).
    - O que observar:
        - Requisições dentro da janela de 10 segundos retornam *cache hit*.
        - Após 10 segundos, a próxima requisição aciona a expiração do cache, forçando o *Edge* a buscar a versão atualizada na Origem *(cache miss)*.

- **Caso 3: Invalidação de cache ativa.**
    - Cenário: O arquivo arquivo30.ext está em cache no *Edge*. Uma alteração ocorre no Servidor de Origem.
    - O que observar:
        - O Servidor de Origem envia uma mensagem via Socket TCP (INVALIDATE /arquivo30.ext) para todos os *Edges* conectados.
        - Os *Edges* removem o arquivo da memória/disco local.
        - A próxima requisição dos clientes força um *cache miss*, garantindo consistência estrita.

Observações

- Data e hora de entrega da tarefa: 20/10/2026 até às 23h59. Portanto, certifiquem-se do arquivo que vão enviar.
- Crie uma pasta chamada “seu-nome-pratica-off1”, comprima e envie pelo SIGAA (única forma de envio).
- Avaliação: o projeto vale 100% da nota da 2ª unidade.
- Os projetos podem ser feitos de forma individual ou em dupla.
- Os projetos devem utilizar as tecnologias vistas na disciplina (*Threads*, *sockets* TCP e UDP).
- Casos de punição com a nota zero, para os envolvidos.
    - Respostas ou códigos iguais.
    - Respostas ou códigos sem a devida explicação.
- Bom trabalho!

Roteiro para Gravação da Apresentação

1. Executar o programa resultante de cada enunciado, questão ou projeto.
2. Para cada programa, execute uma operação por vez.
    - Durante a testagem de cada funcionalidade, comente sobre os possíveis erros e exceções que podem ocorrer mediante entradas incorretas. Comente também sobre quais erros e exceções seu programa captura.
    - Cite as ferramentas e tecnologias utilizadas.
    - Seu programa implementa todas as funcionalidades requeridas? Em caso negativo, justifique.
3. Após a demonstração de funcionamento, é hora de abrir o código.
    - Explique a organização das respostas.
    - Houve alguma dificuldade ou detalhe interessante na implementação de algum dos requisitos mencionados?
    - Quais as principais dificuldades e lições aprendidas com o desenvolvimento do projeto?
    - Responda as perguntas detalhando o código que você implementou.

4. Detalhamento das instruções:
   - O vídeo deve ter no máximo 8 minutos.
   - A gravação será o instrumento avaliativo.
   - Pontos de avaliação:
     - Sequência lógica e domínio do conteúdo (introdução, divisão organizada da apresentação, concatenação de ideias e conclusão).
     - Precisão na comunicação (colocação e entonação da voz, ritmo, dicção e linguagem).
     - Capacidade de argumentação.
     - Domínio e uso de material (organização no uso de recursos de apresentação).
5. Sugestões de software para gravação:
   - Google Meet, Zoom, SimpleScreenRecorder, OBS, etc.